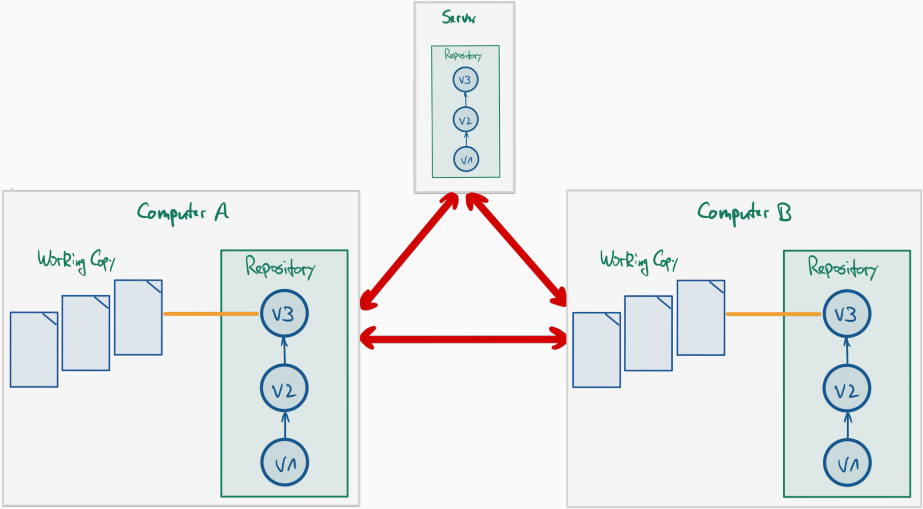
Zusammenarbeit: Git-Workflows und Merge-/Pull-Requests

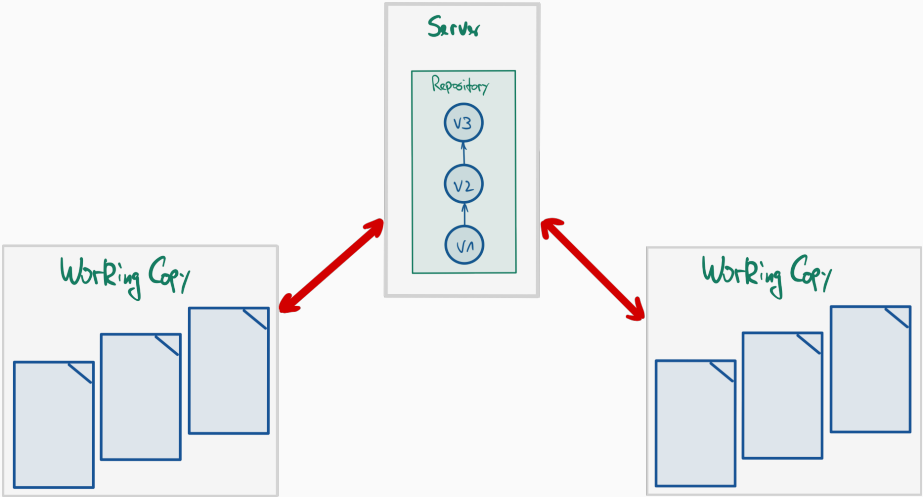
Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

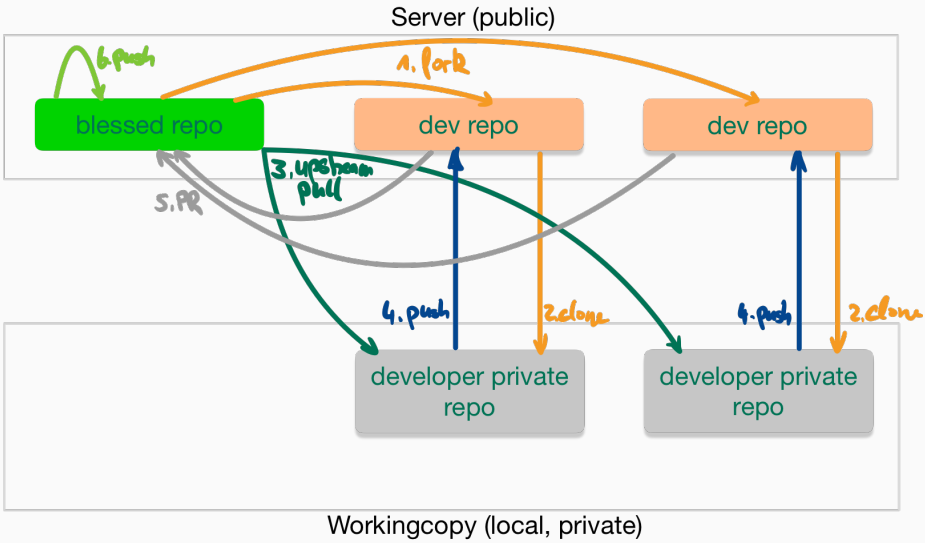
Nutzung von Git in Projekten: Verteiltes Git (und Workflows)



Zusammenarbeit: Zentraler Workflow mit Git (analog zu SVN)



Zusammenarbeit: Einfacher verteilter Workflow mit Git



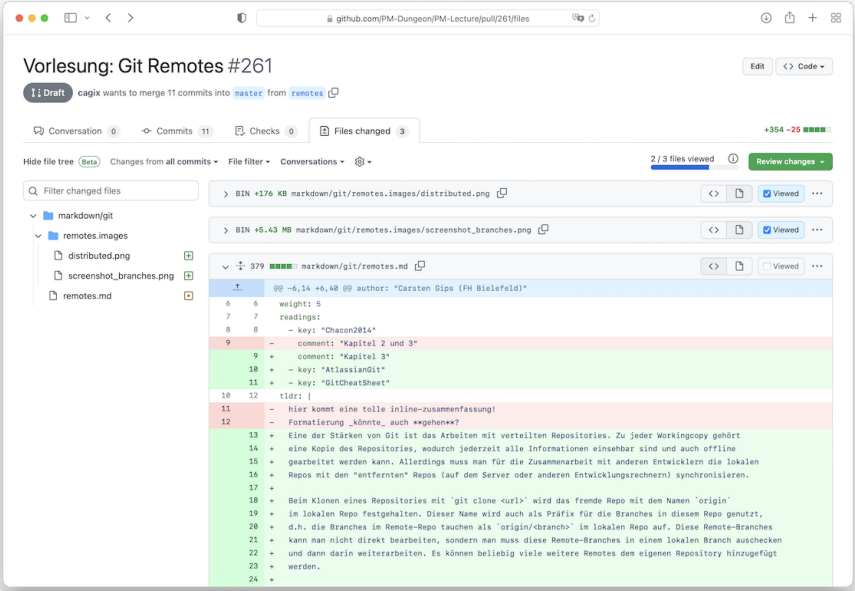
Beispiel: Forken mit Gitlab, Namen für Remotes

Feature-Branches aktualisieren: Mergen mit *master* vs. Rebase auf *master*

```
$ git checkout master          # Workingcopy auf master
$ git pull upstream master     # Aktualisiere lokalen master mit master aus Vorgabe-Repo
$ git checkout feature         # Workingcopy auf feature
$ git merge master             # Aktualisiere feature: Merge master in feature
$ git push origin feature      # Push aktuellen feature ins Team-Repo
```

Besser einen Pull/Rebase für den Feature-Branch machen!

Kommunikation: Merge- bzw. Pull-Requests



Wrap-Up

- Git-Workflows für die Zusammenarbeit:
  - einfacher zentraler Ansatz für kleine Arbeitsgruppen vs.
  - einfacher verteilter Ansatz mit einem “blessed” Repo

- Aktualisieren Ihres Clones und Ihres Forks mit Änderungen aus dem “blessed” Repo
- Unterschied zwischen einem Pull/Merge und einem Pull/Rebase
- Erstellen von Beiträgen zu einem Projekt über Merge-Requests
  - Welche Commits werden Bestandteil eines Merge-Requests (und warum)
  - Diskussion über den Code in Merge-Requests

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Last modified: *15ee831 (lecture: improve wording on feature branches (Git Workflows), 2025-05-08)*